

计算机组成

微机简介

1. 冯诺依曼架构分为input output Memory ALU CU五部分，前两个叫IO，后两个叫CPU

2. 几个层级

- Microprocessor**微处理器**：就是集成的CPU，包含ALU，CU，register；不包含RAM，ROM和IO port
- Microcomputer**微型计算机**：包括CPU，Memory（RAM，ROM），IO port（用来连接外设），BUS（**addr,data,control**）
- Microcomputer system：由微机+软件+IO device构成

3. 硬件介绍

- **CPU**

- **ALU**：用来计算加减乘除和逻辑运算的单元（两个input，通过control signal决定怎么算，得到结果存在register）
- **CU**：在指令下运行，包含一个instruction decoder和一个program counter（也就是IP指向下一条指令的地址）
- **指令集**：分为CISC&RISC，前者指令长度和执行时间不固定，而且是向后兼容的；后者指令fixed on 1 word，容易pipeline
- $min < |DB|, |CPU| > = wordsize$

- **memory**

- hierarchy：register—cache—primary memory—secondary memory
- 上述是由快到慢，但是由于数据是locality的，所以可以预装载让访问速度和大小平衡
- bit:一个二进制数位；Byte=8bits；Nibble=4bits；Word见上面

- **BUS**

- 是一个共享的传输媒介，系统总线就是用来链接处理器，主存，IO的
- Arbitration仲裁：分为分布式的distributed protocols&中心化的centralized scheme

- 分为专用的总线 (**dedicated**) 和复用的总线 (**Multiplexed**)
- 几种bus的接法
 - single-bus: 优点是简单, 缺点是系统吞吐差
 - CPU-central Dual-Bus: 优点是数据传输更高效, 缺点是CPU poor performance
 - Memory-central Dual-Bus: 更好的cpu performance更好的吞吐
- 三种系统总线
 - Data Bus: 是双向的可读可写, 决定着字长; 宽度一般和CPU一样 (8088不一样)
 - Address Bus: 是**单向的**, n个宽度就对应着 2^n 个可寻址单元 (一般是一个字节)
 - Control Bus: 是**两个单向的状态** (command&state)
- **I/O Addressing Scheme IO寻址方式**
 - Memory-mapped IO: IO和memory用同一块address space, 也就是说两者的instruction是一样的, 只需要一个译码电路选出来就行
 - Isolated IO: 可以复用ADDR Bus, 但是也要M/~IO来选择; IO是单独分出来的一块地址

IO Modules&Memory

Memory

1. 基本概念：

- 位置：CPU里的是register；Internal的有cache，main memory（通常是传输一个word）；External的有disk，tape，dvd（通常是传输一个block）
- **Addressable Unit**：可寻址单元，最小的可以addressed的地址，一般internal memory是**BYTE**，Disk是**cluster**
- Access Method：sequential如磁带，direct如磁盘，random如RAM，ROM（访问时延是固定的），associative如cache（数据是根据内容located的）

2. 几种半导体memory

- **RAM** 可以读写，但是是**volatile** 是易失的，需要有电，是暂时的存储
 - **DRAM(dynamic)** 数据就存在电容的电压，需要refreshing。更简单更慢，用在主存
 - **SRAM(static)**数据靠开关的状态存储。更复杂更快，用在cache
- **ROM** 只读，是永久存储不需要电源（用在出厂做好的subroutine，BIOS，函数table）
 - 出厂写好的
 - Programmable Once：可以写一次的
 - EPROM（可以被超高压擦除），EEPROM（可以电力擦除，但是时间更长），Flash如U盘

3. Chip Organization

- 可以做成方形的，优点是减少了pin，每当增加一个pin，capacity*4；缺点是速度下降因为行列要各取一次 / CAS RAS 用来选行列 /
- bit-extension：比如说8片256K*1的可以扩展为256K*8，这个时候address unit是一个word
- word-extension：比如说256K*8想变成2M*8，就得用到译码器；当然如果要求按字节寻址的话，就得要将低几位也拿来译码（但是字长可能变成1个字节）
 - 如果M_cpu>8，又想按字节寻址就得把低log(M/8)拿来译码

I/O module

1. Input Output Techniques

- **programed IO (程序控制IO)**

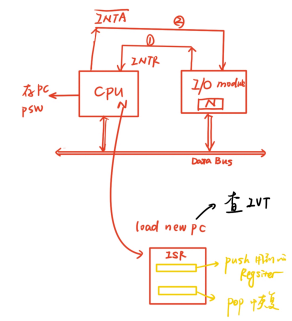
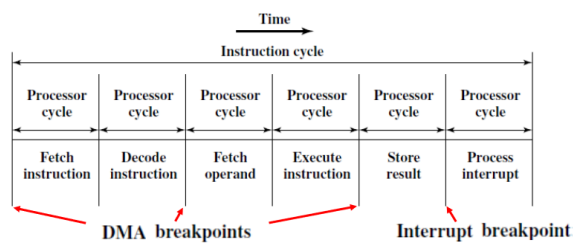
- CPU通过程序来控制IO，就是一次又一次查询设备状态，直到准备好后再开始读写。期间**CPU一直等待外设准备；外设不会主动inform，只是被动等待**
- CPU发出地址寻找IO module——CPU发出控制信号——CPU查询状态——读写

- **Interrupt Driven IO (中断式IO)**

- CPU请求IO操作——CPU不用等待，IO module 准备好后产生中断——CPU在**每一个instruction的最后检查INTR**——如果中断，保存PSW，PC；处理中断；恢复现场
- 确定是哪个外设产生中断的方法 (**bcd也定义了优先级**)
 - a. 每个module都有单独的pin (ARM架构)
 - b. software poll软件轮询：共用一个INTR，CPU每次都去ask到底是哪个产生的，处理之后clear
 - c. hardware poll硬件轮询：和上面差不多，只是用电路去查
 - d. Bus Master：在产生中断之前必须申请bus
 - e. Interrupt controller：8259
- **Localize Handler Programs**
 - general ISR，每次interrupt就进入然后software poll。**缺点是扩展性差**
 - Interrupt vector table，用一个fixed的位置来存中断向量号对应的PC。CPU给外设~INTA之后module通过data bus给CPU是几号中断

- **DMA (direct memory access)**

- DMA是额外的hardware，接管CPU的IO
- CPU告诉DMA controller I/O? address，memory里面的开始地址，需要多少数据；CPU去做其他事情，DMA搬运；DMA结束IO之后发送interrupt给CPU
- DMA会偷走总线周期（一个instruction cycle里面最多三次），每当被偷的时候，DMA只是**transfer A WORD**



◦ 几个cycle的区分：

- clock cycle：就是晶振产生的周期
- bus cycle：CPU用总线完成一次读写的时间
- instruction cycle：见上图

◦ DMA configurations

- 第一种是single bus, detached DMA，每次transfer要占用两次bus（从mem到ctrl；从ctrl到IO）
- 第二种是single bus, integrated DMA，用了更多DMA（更贵），只占用一次bus（从mem到ctrl）
- 第三种是separate bus，只要一个DMA，只占用一次bus（从mem到ctrl）

8086 Microprocessor

1. 8086&8088

- 8086是第一个16bits的处理器有20位地址，即有 $2^{20} = 1MB$ 内存。可以进行流水(pipeline)
- 8088的data bus在内部是16bits的，在外部是8bits的
- 有两种工作模式minimum mode是单个cpu，控制信号来自cpu；maximum mode是多个cpu

2. 8086的结构

- BIU (bus interface unit)：为了数据在CPU和IO，mem之间传输
 - 内部含有**CS,DS,ES,SS,IP**五个寄存器以及一个6bits的指令队列
- EU (execution unit)：为了指令执行
 - 内部含有**AX,BX,CX,DX**(这四个可以分为两个8位的)
 - **SP,BP**(用于stack),**SI,DI**、**FR**(只使用了9/16位)、**ALU**
- 以上两个部分可以并行 (simultaneously)，当EU执行指令，BIU会fetch指令直到队列满
- 其中数据段没有特定的offset可以使用**BX、SI、DI**保存
- pipeline的时候会有branch penalty (预取的指令无用)

3. 8086的工作

- **control signals**

MN/~MX是选minimum mode；~RD告诉外界要读；~WR告诉外界要写；

M/~IO告诉外界是进行memory/IO操作；READY告诉CPUmemory/IO准备好数据了；

~DEN用来使能data transceiver；DT/~R告诉data transceiver数据流向；

~BHE当是0的时候用高八位；ALE是latch的使能信号

HOLD,HLDA是用于DMA要总线的时候；INTR,INTA用于中断；NMI不可以mask的中断

RESET：IP DS SS ES全部变成0，但是CS变成全1。——落在BIOS内

- **BUS Cycle**

- 至少要4个clock cycle
- t1用来放地址；t2用来从输入接脚转换输出接脚的准备；t3用来放data；t4用来recover
- 如果t3之后外界没ready，则会多出wait time，直到准备好
- **Programing&segment**
 - 是按照段的概念**CS:IP, DS(ES):SI/DI/BX, SS:SP/BP**来存储的
 - **physical addr = segment*16+offset**——一个段最多64KB，起始地址是16整数；如果超出了直接wrap around到开头
 - code segment: **CS:IP** 当超出cs段的时候要改CS值**但是8086无法直接修改**可以JMP/CALL
 - data segment:
 - **DS:BI/DI/BX**当超出之后可以直接更改DS的值；8086是little endian，也就是小字节放在小地址（感受起来就是反过来的）
 - **data alignment**: 对齐字的意思是字的起始地址是字的大小的整数倍
 - stack segment: **SS:SP/BP**绝大多数寄存器（除了CS SS DS ES和SP）都能通过push pop来存储。同时它是grow downward的**为了保护其他程序**
- **memory map of 8086**
 - 从上到下640K的RAM(有DOS)，128K的显存，256K的ROM(有BIOS，driver)
 - BIOS的作用：开机的时候检查**有没有错误，从磁盘load DOS**，将控制权给DOS
- **Flag Register**
 - **conditional flags**
 - CF: **有carry或者borrow**的时候为1（d7进位或者d15进位）**针对无符号数**
 - PF: 奇偶校验，**只看低字节，凑成奇数（odd parity）**
 - AF: 当从d3到d4有进位的时候为1，用于BCD加法
 - ZF: 当是0的时候为1
 - SF: 和最高位相同

- OF：当溢出的时候为1（**d6到d7有进位或者d7进出去，but not both**）针对**signed**
- **control flags**
 - IF只有当它为1的时候才允许中断，同时记得重启的时候一定会clear，所以要手动STI，还有进入中断ISR的时候也会clear
 - DF：用来指出string操作的方向
 - TF：用于debug
- **8086的寻址模式（就是MOV的几个用法）**
 - register：MOV BX,DX
data可以在所有register之间传输（除了CS，IP）
 - Immediate：MOV AX,2025H**立即数不能放到segment寄存器**
 - direct：MOV DL,[2025H]，默认是**DS段的偏移**，需要花一个bus去取数据
 - register indirect：MOV AL,[BX]，默认是DS段的偏移，中括号里面的寄存器是DI,SI,BX
 - based relative：MOV AL [BX]+5，默认**BX是DX，BP是SS**
 - indexed relative：MOV DX,[SI]+5，默认是DS段，但是只有这两个index寄存器BI，DI，也可以直接加立即数
 - based indexed relative：MOV CL,[BP][DI]+5就是上面两种加起来，还是看BX，BP来分段
 - segments overrides：MOV SS:[BX][DI]+2,AX **可以强行指定段**

汇编语言

1. 总览：汇编语言分为assembly language instructions和directives，前者是真正的指令，后者可以看作cpp里面的预编译(告诉assembler怎么翻译)，也就是说**后者不占内存**

2. 汇编语言的基本格式

- [label:] + mnemonic [operands] [;comments]
 - 关于label的命名规范：可以用0-9, ?, . @ _ \$这几个，然后第一位不能是数字，整个长度要小于31
 - ; 就是告诉计算机滤过这一行后面的所有东西
- 汇编程序的两种风格
 - 简化的segment definition
 - 只能定义三个段，**.CODE, .DATA, .STACK**，会**自动**链接到CS，DS，SS。DOS会自动设置CS，SS，但是**DS/ES需要手动初始化**
 - 段里面的名字标记的是offset地址，最后面的END MAIN表示程序的进入点是MAIN在的label（**下面这个图就是标准模板**）
 - Full Segment定义
 - 可以定义任意多个段，DOS会自动设置CS，SS，但是要**assume**告诉操作系统，**DS/ES需要手动初始化**
- model definition的大小，要先声明分为
 - SMALL: code <=64KB, data <=64KB
 - MEDIUM: data <=64KB, code >64KB
 - COMPACT: code<=64KB, data >64KB
 - LARGE: data>64KB but single set of data<64KB, code>64KB
 - HUGE: data>64KB, code>64KB
 - TINY: code + data<64KB

3. 汇编语言的基本语法

Control Transfer Instruction

- Range:

- 同一个code segment里trans的: short (ip只改变一个byte), near (ip改变两个byte)
- 跨code segment trans的: FAR, 在call的时候要 RETF)
- 两种方法
 1. **Jumps** (是永远不会自己跳回来的, 必须自己设置)
 - conditional (有很多很多种看flag register) 格式是(JNZ+label)
 - 是 **short jumps**

Mnemonic	Condition Tested	“Jump IF ...”
JA/JNBE	(CF = 0) and (ZF = 0)	above/not below nor zero
JAЕ/JNB	CF = 0	above or equal/not below
JB/JNAE	CF = 1	below/not above nor equal
JBE/JNA	(CF or ZF) = 1	below or equal/not above
JC	CF = 1	carry
JE/JZ	ZF = 1	equal/zero
JG/JNLE	((SF xor OF) or ZF) = 0	greater/not less nor equal
JGE/JNL	(SF xor OF) = 0	greater or equal/not less
JL/JNGE	(SF xor OF) = 1	less/not greater nor equal
JLE/JNG	((SF xor OF) or ZF) = 1	less or equal/not greater
JNC	CF = 0	not carry
JNE/JNZ	ZF = 0	not equal/not zero
JNO	OF = 0	not overflow
JNP/JPO	PF = 0	not parity/parity odd
JNS	SF = 0	not sign
JO	OF = 1	overflow
JP/JPE	PF = 1	parity/parity equal
JS	SF = 1	sign

- unconditional jumps
 - 基本格式是**JMP [SHORT|NEAR|FAR PTR] labels**
 - 默认是near
 - 有关label的补充 (eg AGAIN LABEL FAR)也可以用作JUMP的FAR
- 2. **call** (基本形式 **CALL procedure RET**)
 - 也分为NEAR和FAR两种range call sub1/call far ptr sub1
 - 类似调用函数, func PROC near/far func endp
 - 一般的函数格式是label PROC [FAR|NEAR] label ENDP, **进入的主程序需要是FAR**
 - program会end之后calls 21H interruption with AH=4CH

Data types&Definition

- ORG: 指示offset的起始地址
- DB(byte), DW(word), DD(2word), DQ(4word): 定义变量
- EQU: 定义常量
- DUP: x DB 6 DUP(23H), 相当于创建了一个6位array 里面的值都是23H
- 有关变量的操作 (**Segment, Offset, Type**)
 - MOV AX, SEG x (取x的segment值放在AX)
 - MOV AX, OFFSET x = LEA AX x (取x的offset值放在AX)
- 有关标号 Implicitly: **AGAIN**; Use LABEL directive: **AGAIN LABEL FAR**
- PTR (相当于强制类型转化)

```
DATA1 DB 10H,20H,30H ;
DATA2 DW 4023H,0A845H
MOV BX, WORD PTR DATA1 ; 2010H → BX
MOV AL, BYTE PTR DATA2 ; 23H → AL
MOV WORD PTR [BX], 10H ; [BX],[BX+1] ← 0010H(不这样会有歧义)
JMP FAR PTR aLabel
```

4. 汇编的算数指令

- 加法 **ADD** dest, src **ADC** dest, src
 - dest可以是reg或者mem; src可以是reg, mem或者一个立即数但是不能同时两个是mem
 - 会改变ZF,SF,AF,CF,OF,PF
 - ADC是带进位的, 如果上一次加法有carry, 就额外加1
- 减法 **SUB** dest src **SBB** dest src
 - dest可以是reg或者mem; src可以是reg, mem或者一个立即数但是不能同时两个是mem
 - 会改变ZF,SF,AF,CF,OF,PF
 - SBB是带进位的, 如果上一次加法有borrow, 就额外减1
 - 计算的时候其实是把src取反, 然后加到dest, 最后**反转carry放到CF, 如果CF=0就是正的结果**
- 乘法 **MUL operand**

- byte*byte: 隐藏的是AL, 最后结果放在AX
- word*word: 隐藏的是AX, 最后结果放在DX&AX
- word*byte: AL拿byte, AH=0, word是operand, 最后结果放在DX&AX
- 除法 **DIV denominator** (除数可以是REG或在mem里的 除数不能是0, 商不能比分配的reg大)
 - byte / byte: 被除数在AL, 商在AL, 余数在AH
 - word / word: 被除数在AX, 清空DX, 商在AX, 余数在DX
 - word / byte: 被除数在AX, 商在AL(max 0FFH), 余数在AH
 - double-word / word: 被除数在DX AX, 商在AX(max 0FFFFH), 余数在DX

5. 汇编程序的逻辑指令

- **AND** dest, src按位与; **OR** dest, src按位或; **XOR** dest, src按位异或
 - 是bit-wise的。dest可以是reg或mem里; src可以是reg, mem或者立即数
- **NOT** operand按位取反, op可以是reg或mem
- **逻辑移位** (dest可以是reg或者mem) **超过1要用CL**
 - SHR dest, times: 0→MSB→...→LSB→CF 或者SHL dest, times
 - 举例用法: ASCII—BCD (BCD分为unpacked (就是前面四位是零不管如9=00001001), packed就是8位表示两个BCD

```

    asc      DB    '23'
    pack DB    ?
-----
    MOV AH, asc
    MOV AL, asc+1
    AND AX, 0F0Fh
    MOV CL, 4
    SHL AH, CL
    OR AH, AL
    MOV pack, AH

```

```

asc      DB  '3'
unpack   DB  ?
-----

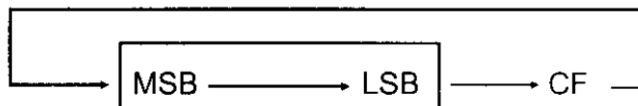
MOV AH, asc
AND AH, 0Fh
MOV unpack, AH

```

- **ROTATE** (旋转, dest可以是reg或者mem)
 - ROR dest times 或者 ROL dest times 只是方向和前面相反



- **ROTATE Cont** (带CF的旋转, dest可以是reg或者mem)
 - RCR dest times 或者 RCL dest times 只是方向和前面相反



- **!!! 以上三种 只有一次的時候可以 instruction xx, 1;**
- **大于一次的时候要 MOV CL, times instruction xx, CL**
- 比较 (**CMP** dest, src)
 - 无符号数的时候 (用 Above/below) **CMP AL, 23 JA label1**

Table 3-3: Flag Settings for Compare Instruction

Compare operands	CF	ZF
destination > source	0	0
destination = source	0	1
destination < source	1	0

Jump Based on Unsigned Comparison

These flags are based on unsigned comparison

Mnemonic	Description	Flags/Registers
JA	Jump if above $op1 > op2$	CF = 0 and ZF = 0
JNBE	Jump if not below or equal $op1 \text{ not } \leq op2$	CF = 0 and ZF = 0
JAE	Jump if above or equal $op1 \geq op2$	CF = 0
JNB	Jump if not below $op1 \text{ not } < op2$	CF = 0
JB	Jump if below $op1 < op2$	CF = 1
JNAE	Jump if not above nor equal $op1 < op2$	CF = 1
JBE	Jump if below or equal $op1 \leq op2$	CF = 1 or ZF = 1
JNA	Jump if not above $op1 \leq op2$	CF = 1 or ZF = 1

- 有符号数的时候

$\text{destination} > \text{source}$ $\text{OF} = \text{SF}$ or $\text{ZF} = 0$
 $\text{destination} = \text{source}$ $\text{ZF} = 1$
 $\text{destination} < \text{source}$ $\text{OF} = \text{negation of SF}$

Jump Based on Signed Comparison

These flags are based on signed comparison

Mnemonic	Description	Flags/Registers
JG	Jump if GREATER $op1 > op2$	SF = OF AND ZF = 0
JNLE	Jump if not LESS THAN or equal $op1 > op2$	SF = OF AND ZF = 0
JGE	Jump if GREATER THAN or equal $op1 \geq op2$	SF = OF
JNL	Jump if not LESS THAN $op1 \geq op2$	SF = OF
JL	Jump if LESS THAN $op1 < op2$	SF $\neq$ OF
JNGE	Jump if not GREATER THAN nor equal $op1 < op2$	SF $\neq$ OF
JLE	Jump if LESS THAN or equal $op1 \leq op2$	ZF = 1 OR SF $\neq$ OF
JNG	Jump if NOT GREATER THAN $op1 \leq op2$	ZF = 1 OR SF $\neq$ OF

- quiz, 把小写字母转化为大写 见PPT
 - XLAT查表指令
 - $\text{BX} \leftarrow \text{OFFSET 表名}$ → 让 BX 指向查表的起始位置（在当前数据段 DS 内）。
 - $\text{AL} \leftarrow \text{索引值}$ （比如 5、12、200 等，必须是 0~255）→ 表示你要查表中第几个字节。
 - XLAT → CPU 自动计算物理地址： $\text{DS}:[\text{BX} + \text{AL}]$ ，从中读取一个字节，存回 AL。
- 所以，XLAT 本质上是一条“ $\text{AL} = \text{表}[\text{AL}]$ ”的快捷指令，前提是 BX 已正确指向表头
- 举个简单的例子就是

```
MOV BX, OFFSET my_table ; BX = 表的偏移地址
MOV AL, 3                ; 想查第 3 个元素 (从 0 开始)
XLAT
```

X86系统中的存储器和IO

1. 内存地址译码(Memory Address Decoding)

- a. **内存地址译码电路**可以译码找到预想的数据存储的地方（通过片选信号找到对应的chip）
- b. **Absolute address decoding**：所有的地址都用来decoding
 - 所有CPU高位地址线均参与对存储单元的译码寻址；低位地址线对芯片内各存储单元的译码寻址
- c. **Linear select decoding**：部分地址参与decoding
 - 部分高位地址线参与对存储单元的译码寻址；存在地址段内容重叠；每个单元有多个地址（aliases）

2. Data Integrity（数据完整性）

- Checksum byte for **ROM**
 - 先把所有bytes加起来，每次都丢掉进位，然后**取补码**
 - 最后检验的时候就是全丢进位加起来，再加上**checksum是0**就行
- Parity bit for **DRAM**
 - even parity：1的总数是偶数（包括parity bit）
 - odd parity：1的总数是奇数（**8086使用的是奇校验**）
- CRC（仅了解）

3. Memory organization in 8086

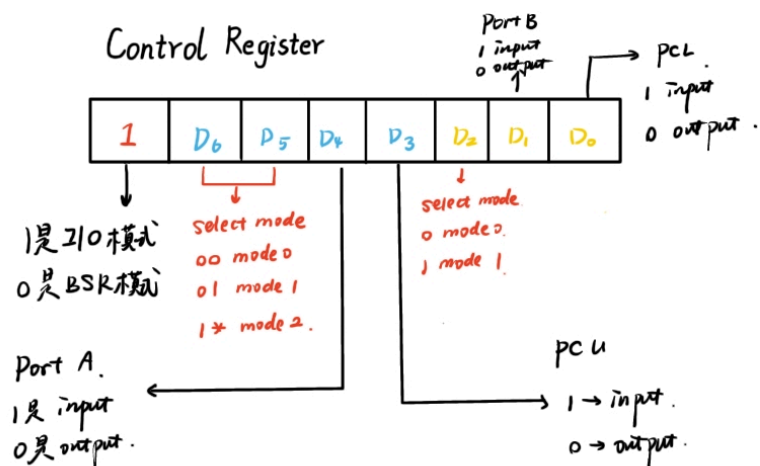
- **Even and odd bank**：
 - addressable unit是字节寻址的，但是对8086可以找到1or2 bytes所以如此设计
 - even存的是D0-D7；odd存的是D8-D15
 - 用A0来选even bank，~BHE来选odd bank
 - 在奇地址取word的时候其实是分成了两步来取的

4. I/O in x86

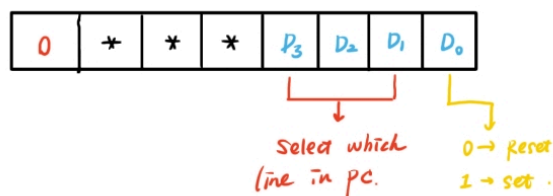
- x86里面的I/O是单独的address space，叫做port只存数据 (**peripheral or isolated I/O**)
 - **I/O instruction (是固定搭配)，只用AL AX**
 - 当port的范围是0-255：
 - 直接 **IN/OUT AL,port 或者 IN/OUT AX,port** (看是8/16bit IO module)
 - 当port的范围是0-65535：
 - 先**MOV DX, port**；然后**IN/OUT AL,DX 或者 IN/OUT AX,DX** (看是8/16bit IO module)
 - 在8086里面，偶地址的port的data在D0-D7；奇地址的port的data在D8-D15 (是design出来的)
-

8255 PPI Chip

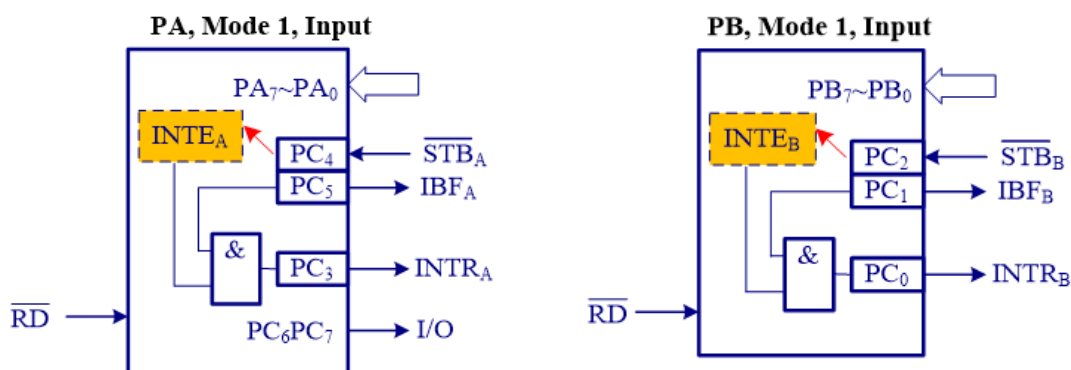
1. 注意看上面提到的要先移到DX !!!
2. 内部结构与接脚
 - 三个data ports：PA0-PA7以及PB0-PB7都可以编程作为I/O；PC0-PC7可以分为**高PCU低PCL**；其中每一位都可以单独拿出来编程
 - **CR** (control register)：用来设置chip；两组Group A (PA&PCU) GroupB (PB&PCL)
 - Data bus buffer，是双向三态8bits的；RESET，high-active，清除CR所有port设置为input
 - A1A0用来选Port&CR；00 A，01 B，10 C，11 CR
 - **8255也能读输出端口，读出的是当前输出值**
3. 操作模式
 - a. CR和操作模式



BSR Mode



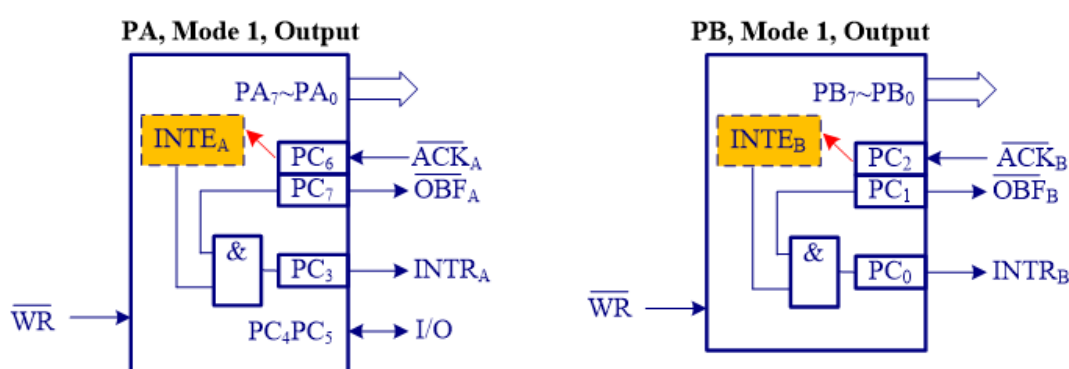
- Mode0 (简单IO模式): PCU(4-7) PCL(0-3)
 - 没有handshaking, CPU可以读或写port通过IN&OUT instruction
 - Input的数据没有锁存, 但是output的数据锁存了
 - Mode1 (strobe I/O) 只有可能靠填空: PCU(3-7)作为PA的handshake; PCL(0-2)作为PB的
 - Input和Output的数据都锁存了
- a. 作为input



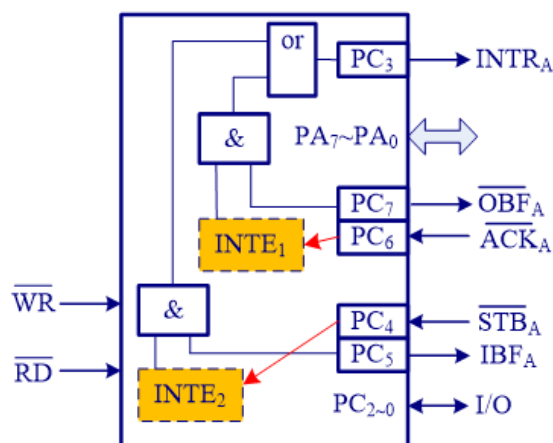
- PC6, PC7还可以单独做IO的line

- $\sim\text{STB}$: Input设备把数据放入latch之后激活
- IBF: 告诉外设现在有数据, 不要再读入
- INTR: 告诉CPU现在有数据, 请求一个interrupt (for interrupt IO);
 $\text{INTR} = \sim\text{STB} \cdot \text{IBF} \cdot \text{INTE}$ (逻辑与)
- INTE: 是CPU的使能, 告诉8255允不允许请求中断
- 一旦CPU取走数据之后IBF和INTR就失活; programed IO就检查IBF和INTR这两个

b. 作为output



- 具体和上面input的功能差不多, 只是还能编程PC4&5
- 当INTR高的时候CPU来写入数据latch之后, $\sim\text{OBF}$ 激活告诉外设来取走数据, 取走之后外设发送 $\sim\text{ACK}$ 告诉8255并且使得 $\sim\text{OBF}$ 高 (失活), 当 $\sim\text{OBF}, \sim\text{ACK}, \text{INTE}$ 都是高电平的时候INTR又来产生中断。
- Mode2 (bidirectional BUS) 只有PA能作为输入输出, PCU(3-7)用来作为握手信号, 输入输出都latch



- PC0-PC2还可以作为单独的IO，检查中断源的时候是看~OBF和IBF谁高谁处理
4. 补充
- 在 Mode 1 和 Mode 2 下，Port C 的各个位保存了握手信号状态。
 - CPU 可以通过执行指令读取 Port C，判断外设状态，从而用**Polling 方式（轮询法）**来控制 I/O，而不一定非得用中断方式。

8253/4 Timer

1. 作用：计数降频。有三个独立的计数器；可以被divided by 1-62536（二进制）或者1-10000（BCD）产生的output有方波/one-shot。

2. 内部结构

- 是三个独立的计数器。**最大计数初值是0**。cpu写入CR（放初值）——搬到CE（开始倒数）——OL（输出当前倒数值，但是需要先latch住）
- 有三态的data bus buffer，~CS用于解码，~RD,~WR用来确定读/写；
- **A1A0**：在写模式下00用于counter0，01用于counter1，10用于counter2，11用于control
- 8254最多10MHz，8253最多2.6MHz
- CE开始当且仅当GATE=1才倒数 **在上升沿检查GATE，下降沿倒数**

3. Control Register

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
SC ₁	SC ₀	RW ₁	RW ₀	M ₂	M ₁	M ₀	BCD

- SC₁SC₀：00的时候选counter0；01的时候选counter1；10的时候选counter2；
- RW₁RW₀：**00的时候发出锁存命令(for OL)**01的时候只读写低字节；10的时候只读写高字节；11的时候先读低再读高
- BCD：是0的时候用16-bits二进制，是1的时候4位BCD

M ₂	M ₁	M ₀	M - Mode
0	0	0	Mode 0
0	0	1	Mode 1
x	1	0	Mode 2
x	1	1	Mode 3
1	0	0	Mode 4
1	0	1	Mode 5

4. 计数模式（都是下降沿开始计数；每次写入的时候都会花一个clock把计数初值从CR搬到CE）

- mode0: **不会自动重复**，当在计数的时候写入新的计数初值的时候会覆盖（但是还是要花一个clock搬运）当set mode之后输出就变低
- mode1: **不会自动重复**，当set mode之后输出还是高，当gate有上升沿的时候开始计数；可以重复触发；新的计数初值不会覆盖，要等到下次gate有上升沿。
- mode2: **会自动重复，n-1个高1个低的重复**，新的计数初值不会覆盖，要等到下一个计数周期；N至少要2（N=1 is illegal）
- mode3: **会自动重复50%的方波**，偶数就是直接每次减2；奇数是high的时候**第一次减1**，之后减2，low的时候**第一次减3**，之后每次减2；也就是 $(n+1)/2$ 的high $(n-1)/2$ 的low；当gate是0的时候OUT会立马变成1，之后重新开始数；当写入新计数值的时候，如果OUT是高就直接开始数，否则等前面数完再开始数。
- mode4: **不会自动重复**，当计数结束之后会产生一个clock的低电平；新的计数初值会直接接在后面开始倒数
- mode5: **不会自动重复**，当gate有上升沿的时候开始计数，计数结束之后会产生一个clock的低电平；当在计数中有上升沿的时候就立马开始重新数；如果触发发生在新的计数之后、终止计数之前，则计数器将在下一个CLK脉冲时加载新的计数。

Interrupt & 8259

1. 8086/8088里的中断

- 一共有256种中断，从00-0FFH

- 其中0-4是预定义好的
- 5-31是给系统用的
- 32-255是给用户的
- 当INT的时候，先push CS, IP, FR; 然后**CS = mem[4*N+2]**, **IP = mem[4*N]**, 执行完之后**IRET**, 将CS, IP, FR都pop回来
- 分类：
 - 硬件中断 INTR: IF=1的时候才使能。**STI/CLI设置/清除IF** , NMI不受IF的影响
 - maskable的中断流程
 - 外设发送INTR给CPU, CPU在每条指令的最后一个时钟周期check, 如果IF=1并且INTR是高的, CPU就发送两个低电平
 - CPU执行ISR N, 首先push FR到堆栈, 然后**自动clear IF and TF**, 然后push CS和IP, load ISR的CS IP, 最后**IRET**
 - NMI: 检查NMI pin (上升沿两个cycle active) 产生INT02中断
 - 软件中断 INT: 不受IF的管理, 遇到就直接去执行ISR程序
 - 预先定义好的:
 - INT 00 (除零错误)
 - INT 01 (单步调试, 当TF=1的时候每个指令之后都会INT 01)
PUSHF POPF就是用来修改Flag register的
 - INT 03 (断点)
 - INTO(04)如果OF是1产生中断, 否则不产生
- 中断优先级 **INT指令>NMI>INTR**

2. 8259和INTR的优先级

- 可以编程改变优先级, by default IR0优先级最高
- 有data bus buffer用来报告中断向量号
- IMR0-7是0的时候不mask
- 还有In Service初始的时候全是0, 当cpu反应之后值1。然后按这个也可以比较优先级

- 执行ISR的时候STI——允许高级中断；EOI——清除IS，允许低级中断；IRET 返回

串行通信和8251

1. 串行通信

- 数据传输率——symbol rate（单位symbol per second=baud）也就是每秒放多少个symbol；Bit rate = baud rate*log₂N（单位是bps，N个symbol）
- 传输方法
 - 异步串行通信：
 - 每次只传一个byte；需要一个开始bit，还可以商量使用parity，stop bits。
 - 要求Tx_C和Rx_C差不多即可（1/8之内）
 - 但是容易畸变，希望中心采样（N即baud factor）让接受时钟比Tx_C快N倍 检测到开始位的时候，**先隔N/2采一次样，然后每隔N采样。**
 - 同步串行通信：
 - 需要把Tx_C的时钟信息编码进去，然后receiver解码出时钟之后开始采样
 - 曼切斯特编码：在数据**1**的时候产生**上升沿**，在数据**0**的时候产生一个**下降沿**
 - character-oriented：用01101000作为SYN字符，01000000作为STX字符检测到1-2个SYN的时候就可以开始采样
 - bit-oriented：用01111111代表空闲状态，01111110作为开始和关闭字符，中间就是每5个1插入一个0来做区分
- 传输模式：
 - simplex单弓只能一个方向；half-duplex半双弓同一时间只能一个方向；full-duplex全双弓，能同时双向传
- 错误检测：
 - parity bit
 - CRC Calculation：就是按照 $M(x)*X^n/G(x)$ 取余数来校验，传输的时候一起传
- 调制和解调：分为调频调幅调相三种方法。QAM编码（就是同时利用A和fai）

2. 8251 USART Chip

- 可以同步/异步；从外面看，只有一个control和一个data。
 - 同步baud rate最多64K，可以是5 6 7 8bits，自动检测插入sync字
 - 异步baud rate最多19.2K，可以是5 6 7 8bits，自动检测插入start stop parity字，TxC和RxC可以是1, 16, 64times of baud rate
- **模式字，控制字，状态字见PPT**
- 由于每次重启之后不确定到底是哪个control register，所有我们采用每次先写三次00，再写一次40H（用来重启）